Sparser MoE Explorations

Router, communication, and systems co-design at 2304 experts

Harry Zhou | Mentor: Robin Zhang

Internship final presentation | July 2026

Presentation outline

Introduction

Sparser MoE Trends

Sparser routing is one possible scaling direction, not a universal design

Open model	Scale	Routed experts	Selected/token
Mixtral 8×7B	Early open sparse MoE	8	2
DeepSeek-V3	671B total / 37B active	256	8
Nemotron 3 Ultra	550B total / 55B active	512	22
DeepSeek-V4-Pro	1.6T total / 49B active	384	6
Kimi K2	1T total / 32B active	384	8
Kimi K3	2.8T total / 104B active	896	16

Trend

LatentMoE reduces the routed representation before expert computation. It provides one path to increase routed-expert count or top-k while controlling the sparse-compute budget.

MoE Execution

MCore carries activations and routing metadata through the sparse MLP

TransformerLayer | MoELayer.forward

Pre-MLP Norm	Router	Latent Down	Dispatch	Expert MLP	Combine	Latent Up
`pre_mlp_layernorm`	`route`	`fc1_latent_proj`	`dispatch_preprocess + dispatch`	`routed_experts_compute`	`token_combine`	`fc2_latent_proj`
Normalize activations $[T \times H]$	Project scores; select top-K	Latent: $H \to H_{\text{latent}}$ Non-latent: 0	Permute routes; exchange inputs	Run local MLPs on received tokens	Exchange outputs; unpermute and sum	Latent: $H_{\text{latent}} \to H$ Non-latent: 0
NORM	**GEMM + METADATA**	**GEMM**	**COMM + META**	**GEMM**	**COMM + META**	**GEMM**
$\mathcal{O}(TH)$	$\mathcal{O}(THE) + \mathcal{O}(TE)$	$\mathcal{O}(THH_{\text{latent}})$	$\mathcal{O}(TKH_{\text{latent}})$	$\mathcal{O}(TKH_{\text{latent}}H_{\text{expert}})$	$\mathcal{O}(TKH_{\text{latent}})$	$\mathcal{O}(THH_{\text{latent}})$
normalized states	routing metadata	latent activations	dispatched inputs	expert outputs	combined latent output	output states

Scope: routed-expert path only; shared-expert execution is omitted.

Optimization target. Sparse performance depends on router work, metadata transformation, token exchange, expert kernels, and weighted recombination.

LatentMoE Cost Model

Latent representations reduce exchange payload, not expert computation

Expert computation

$$\mathrm{FLOPs}_{\mathrm{expert}} \propto TKH_{\mathrm{latent}}H_{\mathrm{expert}}$$

Every selected route still executes its expert MLP. On a non-latent path, down/up projections are absent (zero cost) and $H_{\mathrm{latent}} = H$.

Token exchange

$$\text{payload} \propto TKH_{\mathrm{latent}}$$

Router score projection costs $\mathcal{O}(THE)$ and top-k selection adds $\mathcal{O}(TE)$ metadata work. Latent MoE narrows the exchanged payload when $H_{\mathrm{latent}} \ll H$; without it, $H_{\mathrm{latent}} = H$.

Sparse-path budget. Selection, metadata transformation, communication, expert compute, and combine remain coupled.

Target Workload

Presentation outline

Target Workload

Experimental Configuration

A Qwen3-Next-derived latent-MoE training model isolates large-expert sparse-path costs

Backbone & Attention

24 layers; $H = 2048$
GQA: 32 Q / 2 KV groups; GDN disabled

Latent MoE

2304 experts; top-$k = 36$; $H_{\text{latent}} = 512$
FFN/shared width 512; EP72; 32 E/rank

4.5×
router scores
512 → 2304

3.6×
selected routes
10 → 36

Recipe

4K tokens; NV72 systems (GB200 / GB300).
TP1 / PP1 / EP72; MBS 3 / GBS 864; MXFP8.
1F1B; paged stash; full-iteration CUDA graph.

Initial Analysis

Presentation outline

Initial Analysis

Initial Profile

A historical snapshot exposed the critical path

Region	Time
Attention	3.1 ms
HybridEP dispatch	9.7 ms
Expert MLP	1.1 ms
HybridEP combine	2.5 ms

Principal bottlenecks

1. CPU launch overhead from many small kernels.
2. Fused-router degradation at large E and K.
3. Communication and preprocessing overhead.

Eliminate launch overhead before analyzing the replay-stable GPU critical path.

Launch-Overhead Mitigation

Make one iteration replayable, then optimize the remaining GPU bottlenecks

```python
# capture once
for microbatch in iteration:
    route()
    dispatch()
    grouped_mlp()
    combine()

# replay without Python launch gaps
cuda_graph.replay()
```

- Increase microbatch size where memory permits.
- Capture the full iteration, not isolated kernels.
- Use fused grouped MLP and related MoE kernels.
- Fuse metadata and dispatcher preprocessing kernels.

Router Optimization

Presentation outline

Fused-Router Optimization

Radix Selection

TE PR #2821 replaces repeated maximum selection with a one-warp radix selector

Repeated maximum selection

A rank-r pass scans E scores and checks up to $r-1$ prior IDs:

$$\sum_{r=1}^{K} E\,r = O(EK^2).$$

At $E = 2304$, $K = 36$, selection repeats a full expert-row walk for every selected rank.

Radix selection

FP32 order bits are narrowed by eight 4-bit passes, followed by a constant number of gathers:

$$8E + 2E = O(E).$$

At $E = 2304$, per-thread selection needs a 9 KB score row per lane. CTA-wide selection adds inter-warp barriers and shared-memory coordination. TE retains one warp per token: lanes stride across experts and use shuffle collectives.

2304
experts
one score row per token

36
selected IDs
large sparse fan-out

8
radix passes
FP32, 4 bits/pass

Radix Selection: Algorithm

Each pass narrows the rank-K score prefix without materializing a sort

```
ordered = float_to_ordered_uint(scores)
prefix, mask = 0, 0
rank = topk

for pass in range(7, -1, -1):
    hist[0:16] = count scores matching prefix
    bucket = highest_bucket_containing(rank)
    prefix |= bucket << (4 * pass)
    mask   |= 0xf << (4 * pass)

gather scores > prefix
gather ties in index order until topk
```

Two phases

1. Count matching scores in 16 buckets; choose the bucket containing rank K.
2. Gather scores above the final value; use ordered ties to complete exactly K IDs.

Invariant

The selected set is exact. Radix changes how the threshold is found, not the top-k semantics or deterministic tie handling.

Radix Selection: Implementation

The CUDA kernel carries the rank state in registers and uses warp collectives

```
constexpr int kPasses = 8;
unsigned prefix = 0, mask = 0;
for (int pass = kPasses - 1; pass >= 0; --pass) {
  unsigned packed[4] = {};
  count_matching_scores(packed, prefix, mask);
  int bucket = find_target(packed, k_remaining);
  prefix |= unsigned(bucket) << (4 * pass);
  mask   |= 0xfu << (4 * pass);
}
```

One warp / token

The implementation maps a token row to a warp. Each lane strides across experts, then warp reductions combine 16 bucket counts.

Bounded support

Packed 8-bit counters support $E \leq 8160$; the 2304-expert target is well inside that range.

Secondary Optimizations

PR #3012 turns the radix algorithm into a throughput-oriented kernel family

Loop fusion

Merge clear, load, score, save, and bias work where the score path permits.

Async load

Double-buffer raw logits and use a persistent grid sized by occupancy.

Packed radix

Store sixteen counters in four 32-bit registers instead of 32 registers.

Static score

Instantiate score functions at compile time to remove hot-path branches.

Merged head

Keep a specialized small-k head while routing large shapes to the optimized head.

Design principle

The algorithmic fix removed the large-K cliff. These refinements remove data movement, register, branch, and occupancy costs exposed after that cliff is gone.

Secondary Optimizations: Loop Fusion

The score path writes each expert value once instead of staging separate loops

```
if constexpr (ScoreFunc == kSigmoid) {
  for (int i = lane_id; i < E; i += 32) {
    float value = sigmoid(raw_logits[i]);
    intermediate[pos + i] = value;
    if (bias) value += bias[i];
    scores[i] = value;
  }
} else if constexpr (ScoreFunc == kSqrtsoftplus) {
  for (int i = lane_id; i < E; i += 32) {
    intermediate[pos + i] = raw_logits[i];
    scores[i] = sqrtsoftplus(raw_logits[i]);
  }
}
```

Fused dataflow

A single strided loop converts logits, evaluates the score function, preserves the value required by backward, applies optional bias, and populates selection input. The softmax path

remains multi-pass because its normalization is mathematically coupled across the entire expert row.

Secondary Optimizations: Async Loading

Raw logits are prefetched with `cp.async` while the current tile is selected

```
// tile n+1 starts before tile n selects
loader.start_load(next_logits, E, lane_id);

loader.wait();
select_topk(loader.current_buf());

// 16-byte global-to-shared copy
cp_async_16B(dst, src);
cp_async_commit();
```

Overlap policy

The loader keeps the original input dtype in shared memory, deferring conversion to compute. It uses 16-byte vector copies on supported GPUs and a scalar fallback otherwise. The host launcher

chooses one or two buffers from the shared-memory budget, then sizes the persistent grid from active blocks per SM.

Secondary Optimizations: Packed Radix

Eight-bit fields lower histogram bookkeeping from 32 registers to four

```
// 16 counters in 4 packed u32 values
unsigned packed[4] = {};
int bucket = (u >> digit_pos) & 0xf;
int pack = bucket >> 2;
int shift = (bucket & 3) << 3;
packed[pack] += 1u << shift;

unsigned count = (packed[b >> 2] >> ((b & 3) << 3)) & 0xffu;
unsigned total = warp_allreduce_sum(count);
```

Why it is safe

Each lane sees at most $\lceil E/32 \rceil$ scores per bucket. For $E = 2304$, the largest count is 72, which fits in an 8-bit field. The implementation

enforces $E \leq 8160$ so a packed counter cannot overflow.

Secondary Optimizations: Static Scores

Compile-time score specialization removes the runtime score-function branch

```
template <typename T, TopkFuncType TopkFunc, int ScoreFunc>
__global__ void router_forward(...);

if constexpr (ScoreFunc == kSoftmax)
  softmax(scores, E, lane_id);
else if constexpr (ScoreFunc == kSigmoid)
  scores[i] = sigmoid(raw_logits[i]);
else if constexpr (ScoreFunc == kSqrtsoftplus)
  scores[i] = sqrtsoftplus(raw_logits[i]);
}
```

Specialization

The launcher performs the score-function switch once, then each instantiated kernel contains only its required math and synchronization. This benefits both top-k and

auxiliary-loss forward/backward paths, where branch structure otherwise repeats for every expert.

Secondary Optimizations: Merged Head

A single launcher preserves a lightweight head for small k and the optimized head for large shapes

```
bool use_radix = topk >= radix_threshold && E <= kMaxExperts;

if (!use_radix)
  launch_simple(simple_kernel<...>);
else
  launch(optimized_kernel<..., Radix, ScoreFunc>);
```

Shape-aware dispatch

The simple head avoids async-loader and persistent-grid overhead on conventional small-k shapes. The optimized head is selected only when radix work dominates. This is why gains at

$2304/36$ do not require regressing common router shapes.

Dense Routing Map

TE PR #3129 replaces the $[T, E]$ byte map with selected int16 IDs

Per-token representation

Format	Shape	Bytes/token
Byte map	$[E]$	2304
Selected IDs	int16 $[K]$	72

$$\frac{E}{2K} = \frac{2304}{2 \times 36} = 32 \times .$$

Target batch

Local metadata, $T = 8192$	Size
Byte $[T, E]$	18.9 MB
int16 $[T, K]$	0.59 MB

The compact buffer records expert IDs only.

Probabilities remain separate because training needs selected weights and their backward dependencies.

Dense Routing Map: Forward

The index-output API emits one int16 ID per selected route

```
for (int i = lane_id; i < topk; i += 32) {
  int expert = topk_indices[i];
  if (indices_out)
    indices_out[token * topk + i] = static_cast<IndexType>(
      expert);
  probs[pos + expert] = scale * topk_scores[i];
}

if constexpr (IndexType == int16_t)
  NVTE_CHECK(E <= INT16_MAX, ...);
```

Producer contract

The selected ID output is optional: legacy bytemap/bitmap callers retain their existing format, while dense-route callers receive $[T, K]$ indices directly from the fused selection result. $E = 2304$ is safely

representable in int16.

Dense Routing Map: Backward

The dedicated index path traverses selected IDs and zero-fills the dense gradient

```
const IndexType* ids = topk_indices + token * topk;
zero(grad_logits + pos, E);

for (int k = lane_id; k < topk; k += 32) {
  int expert = static_cast<int>(ids[k]);
  auto grad = grad_probs[pos + expert] * scale;
  grad_logits[pos + expert] = grad;
}
```

Complexity change

Replace repeated membership tests of every expert against a K-entry list with a dense zero-fill plus direct writes to selected locations:

$$O\big(T(E + K)\big) \text{ instead of } O\big(TEK\big).$$

Full-row score-function coupling still performs the mathematically required row work; the redundant membership scan is removed.

Router Bounds

Optimistic limits contextualize the measured 254.5 μs result

HBM traffic floor

Semantic traffic is approximately 245 MB:

$$245 \text{ MB}/8 \text{ TB/s} = 30.7\,\mu\text{s}.$$

The measured kernel reaches about 12% of raw HBM bandwidth. Tensor bytes only; cache-line and instruction overhead are excluded.

Compute-issue floor

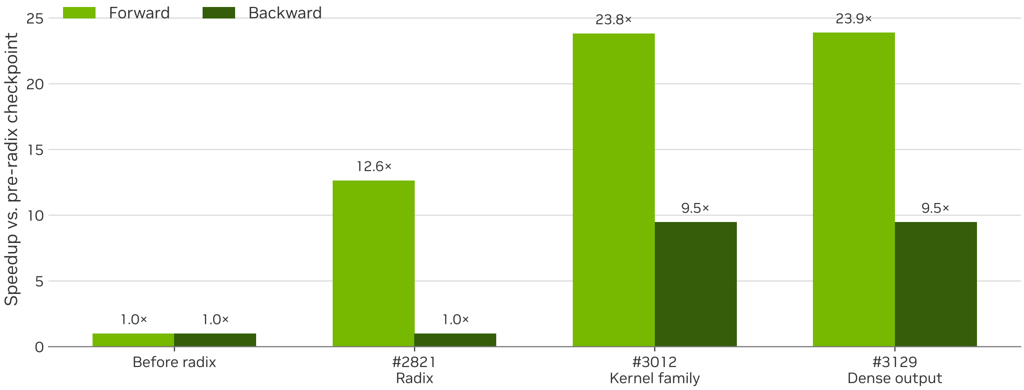
Eight radix passes touch 151M scores:

$$8 \times 8192 \times 2304 = 151\text{M}.$$

One FP32 op/visit at 75 TFLOP/s/GPU gives an impossible $\sim 2.0\,\mu$s floor. Masks, shifts, histograms, shuffles, scans, and dependencies remain.

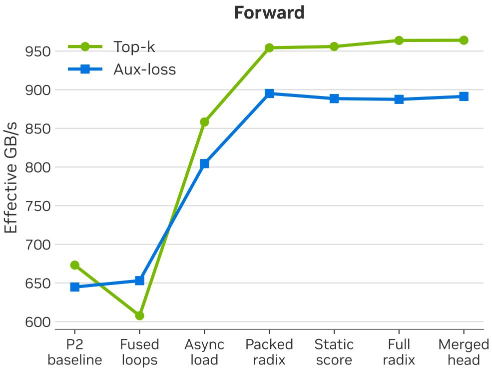
Router Stages

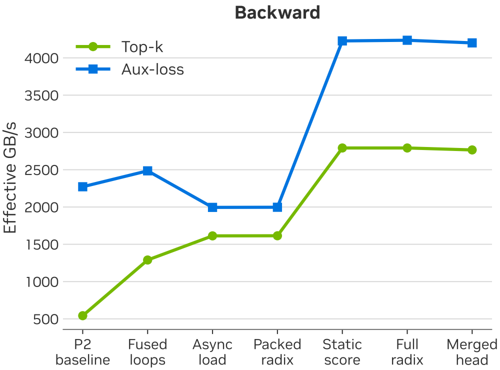
Matched B300 trace-back: 65536 tokens, 2304 experts, top-$k = 36$, softmax



Secondary Gains

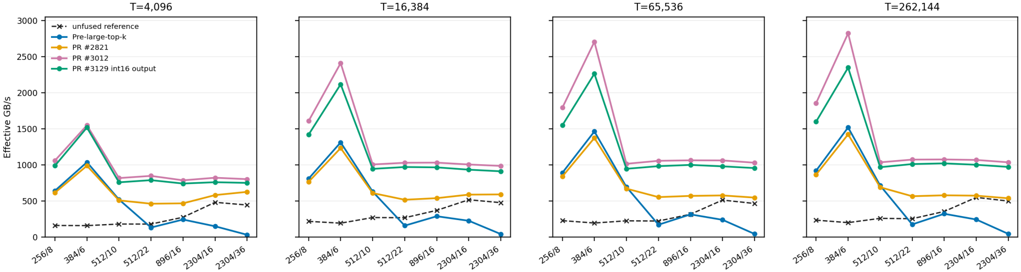
Incremental 8192-token result for the target 2304/36 shape





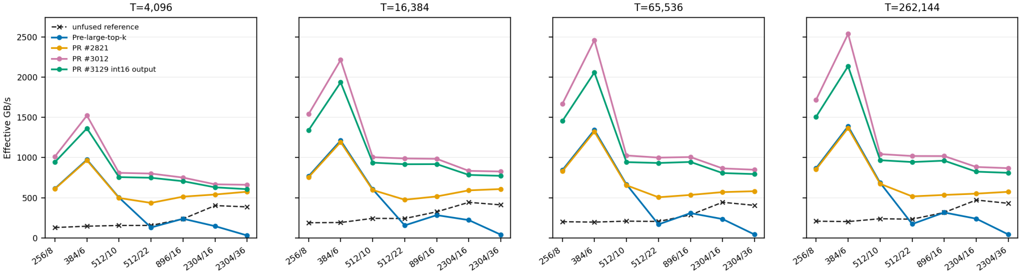
Top-k Forward

0730 B300 trace pass: softmax; seven router shapes and four token counts



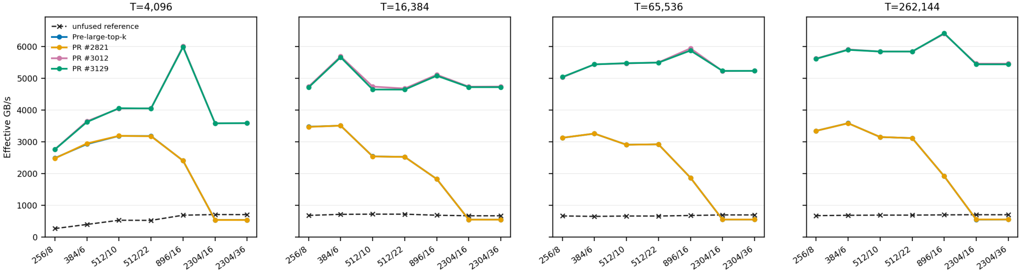
Top-k Forward

0730 B300 trace pass: sigmoid; seven router shapes and four token counts



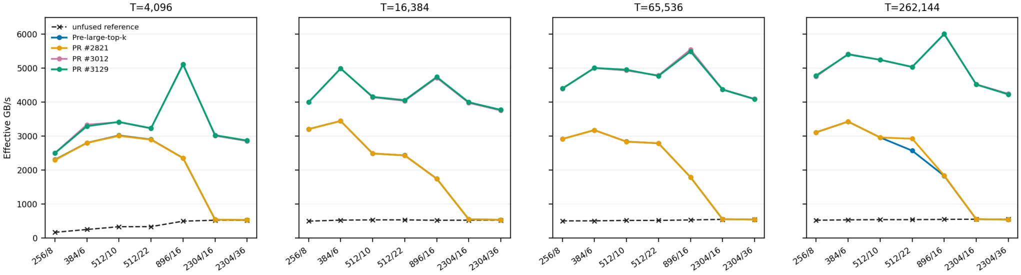
Top-k Backward

0730 B300 trace pass: softmax; seven router shapes and four token counts



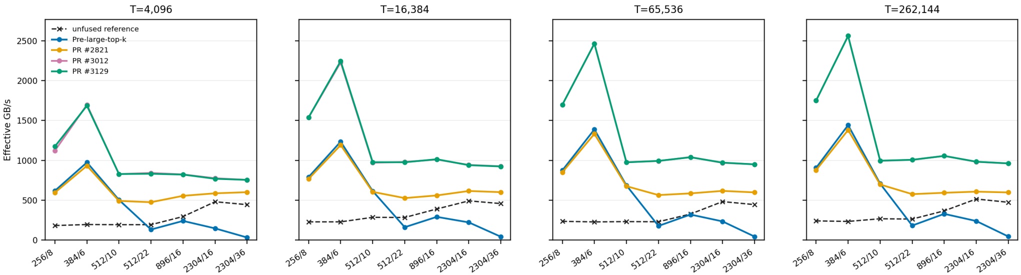
Top-k Backward

0730 B300 trace pass: sigmoid; seven router shapes and four token counts



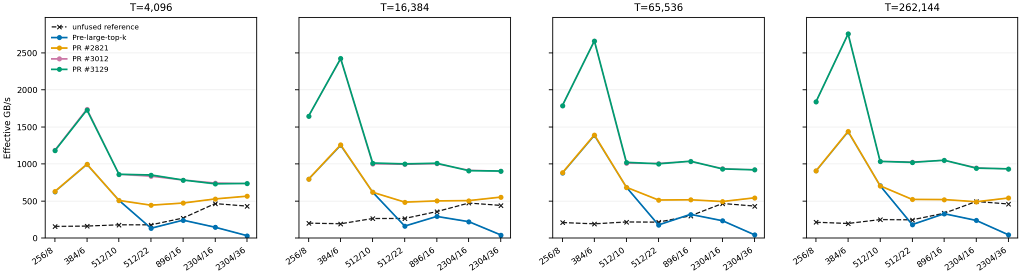
Aux-loss Forward

0730 B300 trace pass: softmax; seven router shapes and four token counts



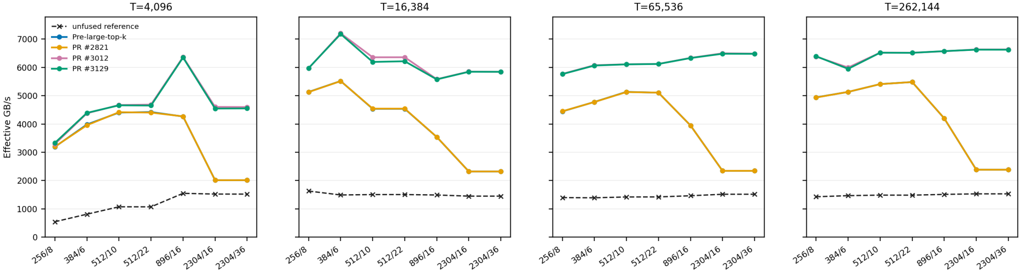
Aux-loss Forward

0730 B300 trace pass: sigmoid; seven router shapes and four token counts



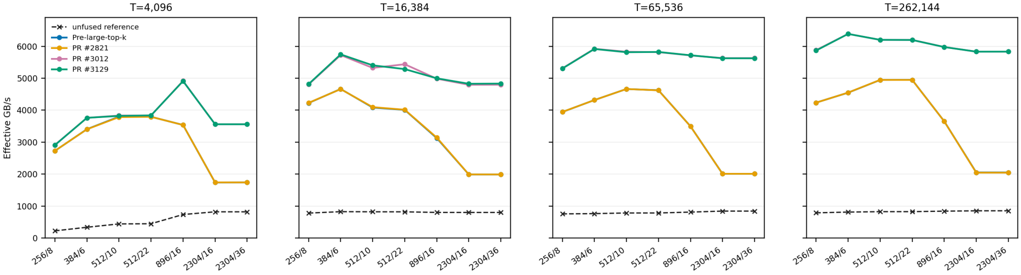
Aux-loss Backward

0730 B300 trace pass: softmax; seven router shapes and four token counts



Aux-loss Backward

0730 B300 trace pass: sigmoid; seven router shapes and four token counts



HybridEP Optimization

Presentation outline

HybridEP Optimization

1. Introduction
2. Target Workload
3. Initial Analysis
4. Fused-Router Optimization
5. HybridEP Optimization
6. MCore Integration
7. End-to-End Results

HybridEP Pipeline

Sparse routing reduces metadata work, local work, and combine latency

Metadata
Route IDs $\rightarrow$ scan/bitsets

Routing
Dispatch $\rightarrow$ permute

Execution and return
Expert MLP $\rightarrow$ combine $\rightarrow$ unpermute

Compact metadata
Avoid bool $[T, E]$ and repeated membership tests.

Sparse local execution
Skip inactive expert slots in permute and unpermute.

Pipeline configuration
Tune pull depth, grouping, and reduction batch jointly.

NVIDIA

Probability Transport

Selected probabilities remain required for weighted activations and gradients

```
# keep mathematical state
ids, weights = router(logits)

# communicate only what this owner needs
for route in selected_routes:
    send(token[route], weights[route])

# backward reuses the selected weights
grad_logits = router_backward(ids, weights,
     grad)
```

103.0 μs
dispatch with probability payload
controlled B300 NVL8

94.4 μs
without prob payload
+9%

249.4 μs
combine with probability payload
pull + reduce

210.0 μs
without prob payload
+19%

The “without probability” rows bound transport opportunity; training still preserves selected weights and their backward dependencies.

Ballot Permutation

4.5 active local experts/token leave 86% of slots empty

```
active = route_map[token * E_local + lane] >
    0;
mask = __ballot_sync(FULL_MASK, active);

while (mask) {
    expert = __ffs(mask) - 1;
    mask &= mask - 1;
    peer = __shfl_sync(FULL_MASK, my_peer,
     expert);
    move_selected_token(peer);
}
```

4.27$\times$
permute
396.1 $\rightarrow$ 92.7 μs

2.04$\times$
unpermute
269.5 $\rightarrow$ 131.8 μs

Threading policy

Use 32 threads/token at latent $H = 512$. Larger hidden sizes retain wider thread groups to expose memory-level parallelism.

NVIDIA

Dense Route Scans

Build expert and rank membership once; use bit tests downstream

```
local_mask = 0
rank_mask = 0

for slot in range(K):
    expert = dense_route[token, slot]
    local_mask |= 1 << (expert % E_local)
    rank_mask  |= 1 << owner_rank(expert)

if local_mask & (1 << expert):
    process_active_expert()
```

1.58×

Isolated 512-thread scan

$525.6 \to 331.7\ \mu\mathrm{s}$

The scan consumes compact $[T, K]$ route IDs directly; bool route-map reconstruction is unnecessary.

No permute-fusion metadata is included in this isolated comparison.

Combine Configuration

Tuned for 2304 experts and validated at EP72

```
for group in [1, 2, 4, 8]:
    for pull_depth, reduce_batch in
     candidates:
        time(combine)
        time(combine + unpermute)
        verify_correctness()

select shallowest queue that hides pull
     latency
validate the selected tuple in the EP72
     launch
```

Parameter	NVL8 study	2304E / EP72
Combine SMs	32	32
G2S stages	64	72
S2G stages	8	8
Tokens/group	2	2
Reduce batch	16	72

The EP72 tuple matches the wider communication domain; neither tuple is universal.

Combine Pipeline

Group 2 balances parallelism with FIFO lookahead

G2S fetch
Remote TMA reads

$\rightarrow$

Shared FIFO
32 slots/pipeline

$\rightarrow$

Reduction
FP32 accumulation

$\rightarrow$

S2G store
Write result

Lookahead

32 FIFO slots / ~ 8 source ranks ≈ 4 prefetched tokens.

Saturation

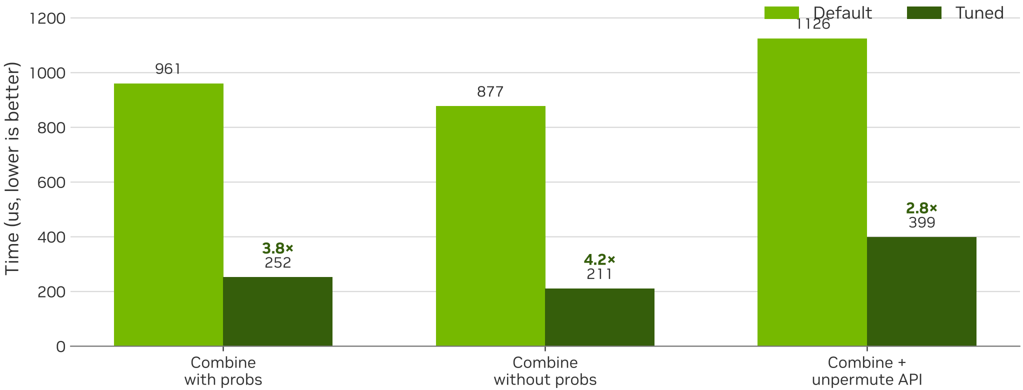
G2S 64 $\rightarrow$ 128 changes 246.1 to 245.0 μs.

SM allocation

Group 2 matches group 1 throughput with 32 rather than 64 SMs.

Combine Tuning

Controlled H=512, local-E=32, top-$k = 36$ comparison



Dispatch vs. Combine

Dispatch is asynchronous; combine synchronizes, reduces, and stores

Dispatch / push

116.8 μs

650 GB/s
58% long-scoreboard stall
0.05% barrier stall

Combine / pull + reduce

254.4 μs

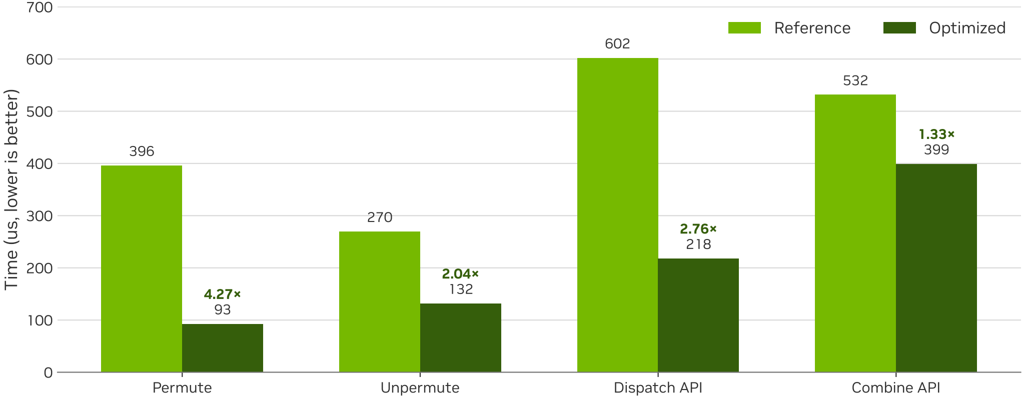
271 GB/s
27% long-scoreboard + 26% wait
15% barrier stall

Separate combine and unpermute

Fusing them serializes the consumer behind slowly produced chunks: 434 μs fused versus 381 μs standalone, about 14% slower.

Microbenchmarks

B300 NVL8, H=512, 8192 tokens/rank, local-E=32, top-$k = 36$



NVIDIA

Custom All-Gather

Direct registered-buffer writes reduce intermediate copies

$\sim$15%

historical isolated gain

custom all-gather vs NCCL

32$\times$

logical metadata reduction

bool $[T, E]$ to int16 $[T, K]$

When it applies

- Writes directly into HybridEP registered buffers.
- Avoids a separate NCCL-buffer-to-PyTorch-tensor copy.
- Benefit is collective- and topology-specific.
- Compact metadata reduces bytes before collective choice matters.

Scope: an isolated communication optimization; the full-model gain is not attributed to all-gather alone.

Design Rejections

Profiling narrows the feasible design space

Direct permute

Repeated remote writes and address work moved the bottleneck into dispatch; the saved 92 μs permute did not compensate.

Fused combine

Producer–consumer serialization left the best fused path 14% slower than standalone.

Warp-pruned scan

Extra bitset initialization and coordination cost more than the skipped ballots on key shapes.

MCore Integration

Presentation outline

MCore Integration

Megatron Core

Integration boundaries

Collaborating workstreams

- Paged stash for bounded activation storage.
- Full-iteration CUDA graph capture and replay.
- 1F1B MoE communication/compute overlap.
- Capture-safe memory release without deferred-free inflation.

Delivered plumbing

```
if te.has_dense_route_output() and \
   dispatcher.accepts_dense_route():
    route = int16[T, K]
    fused_router(..., route_out=route)
    hybrid_ep(..., dense_route=route)
else:
    compatibility_bool_route()
```

Capability detection preserves backward compatibility.

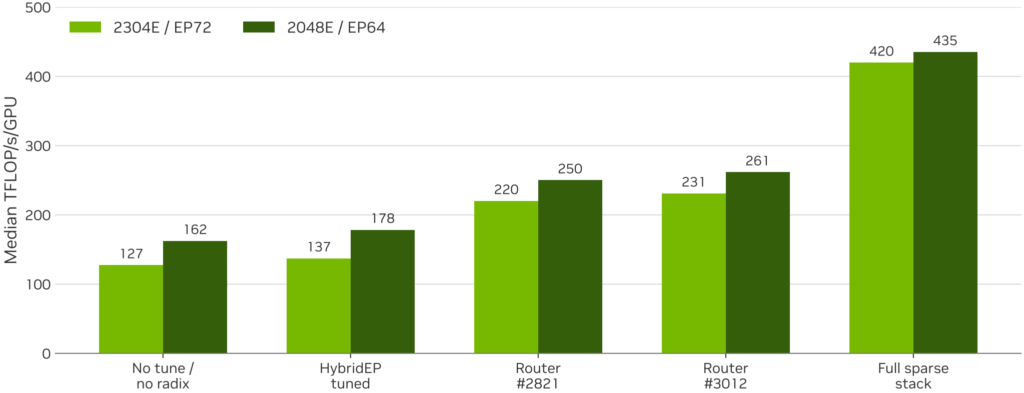
End-to-End Results

Presentation outline

End-to-End Results

Full-Model Stages

OCI-AGA GB300, 1F1B + paged stash + full-iteration graph; median after warmup 5



Full-Model Throughput

Cross-layer optimization: router, compact routes, sparse preprocessing, bitsets, and tuned combine

419.7
2304E / EP72 median TFLOP/s/GPU
127.2 no-tune baseline; 3.30×

435.1
2048E / EP64 median TFLOP/s/GPU
162.2 no-tune baseline; 2.68×

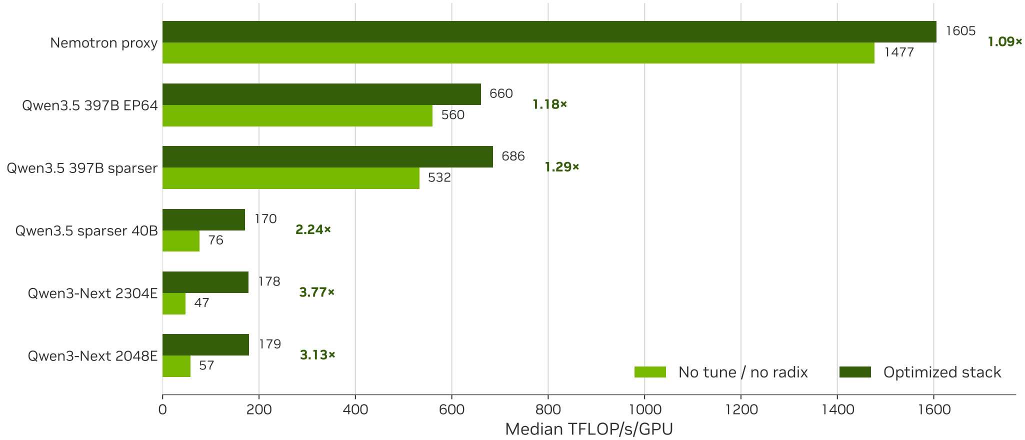
530 / 423
iterations observed
2304E / 2048E full-stack runs

Matched stack

Both shapes use the same 26.06 image family, mock data, MBS 3, 24 layers, full 1F1B, paged stash, full-iteration graph, and the wgrad fix that disables the CuTe DSL fused GroupedMLP wgrad subpath. Isolated kernel speedups are not added together.

MoE-Only Matrix

OCI-AGA GB300; 155 post-warmup iterations; matched full graph + paged stash



MoE-Only Scaling

Generalization across wide, sparse expert shapes

3.77×
Qwen3-Next 2304E
47.2 → 177.9

3.13×
Qwen3-Next 2048E
57.4 → 179.4

2.24×
Qwen3.5 sparser 40B
76.1 → 170.3

Limiting case

Nemotron's proxy shape improved only 1.09×: when expert compute is already large, router and sparse communication are a smaller fraction of the module. This matrix is MoE-only, without 1F1B; full graph and paged stash are matched.

Conclusions

Key findings and applicability limits

Findings

- Radix selection resolves the pathological large-E, large-K case.
- Compact int16 routes reduce logical metadata $32\times$.
- Ballot skips inactive local experts.
- Combine performance requires coordinated queue, group, and batch tuning.
- Gains generalize most strongly to sparse, wide expert shapes.

Scope

- Do not aggregate isolated speedups into end-to-end predictions.
- The EP72 tuple is not a universal default.
- Training retains selected probability transport.
- Separate this work from graph, stash, and 1F1B contributions.
- Distinguish MoE-only and full-model measurements.

Sparse Path

Selected routes remain compact from routing through replay

Select
TE fused router

Radix + persistent

Represent
int16 $[T, K]$

Avoid $[T, E]$ expansion

Exchange
HybridEP

Slice probabilities; bitsets

Pipeline
Combine

Tune stages / groups / batch

Replay
Megatron Core

Capture-safe full stack

23.9×
router forward
pre-radix → dense output

1.84×
HybridEP API total
1133.7 → 617.1 μs

419.7
full-model TFLOP/s/GPU
2304 experts / EP72

Appendix

Presentation outline

Appendix

8 Appendix

Combine Sensitivity

EP72: group 2 retains throughput with half the SMs

Configuration	Time	Output BW	Interpretation
Group 1 / 64 SMs	252 μs	265 GB/s	Full FIFO depth; only half of warp resources are utilized
Group 2 / 32 SMs	254 μs	263 GB/s	Same throughput with half as many SMs
Group 4 / 32 SMs	502 μs	133 GB/s	Insufficient FIFO depth to hide pull latency
G2S 64 $\rightarrow$ 128	246 $\rightarrow$ 245 μs	flat	Additional queue depth provides no measurable gain

Router Timings

65536 tokens, 2304 experts, top-$k = 36$, softmax

Checkpoint	Forward	Backward	Output
Before large top-k support	45.358 ms	3.552 ms	sparse bool
PR #2821	3.587 ms	3.553 ms	sparse bool
PR #3012	1.904 ms	0.375 ms	sparse bool
PR #3129	1.899 ms	0.375 ms	dense int16

Interpretation

PR #2821 changes the selection algorithm; PR #3012 redesigns forward and backward; PR #3129 changes the API representation without regressing the target forward path.

Sources

The presentation is reproducible from local measurements and official model sources

Topic	Primary source
Persistent content memory	`../sparser_moe_final_presentation_content.md`
Router trace-back	0730 trace-back checkpoints; incremental optimization records
HybridEP	`hybrid-ep-sparse-opt-new.md`; `hybrid_ep_rebased_bw_results.md`
Dense scan	`isolated-scan-bench.md`
Full-model sweep	`qwen3_next_2606_image_sweep_20260728/README.md`
MoE-only matrix	`moe_perf_canonical_matrix_20260730/README.md`
Open models	Mixtral; DeepSeek-V3; Nemotron 3 Ultra; DeepSeek-V4-Pro; Kimi K2; Kimi K3
Hardware roofs	NVIDIA HGX B300; NVIDIA HGX AI Factory components

Discussion